

国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第七章： 控制器设计的经典方法

Chapter 7 Classical Controller Design Methods



Professor Dingyu Xue, xuedingyu@mail.neu.edu.cn
School of Information Science and Engineering,
Northeastern University, Shenyang, CHINA



本章主要内容

- 已知受控对象模型，如何为其设计控制器，得到满意的控制效果？
 - 超前滞后校正器设计方法
 - 基于状态空间模型的控制器设计方法
 - 最优控制器设计——OCD
 - 控制系统工具箱的设计界面
 - 多变量系统的频域设计方法
 - 多变量系统动态解耦

国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第七章 控制器设计的经典方法

超前滞后校正器设计方法

Design of Phase Lead/Lag Compensators

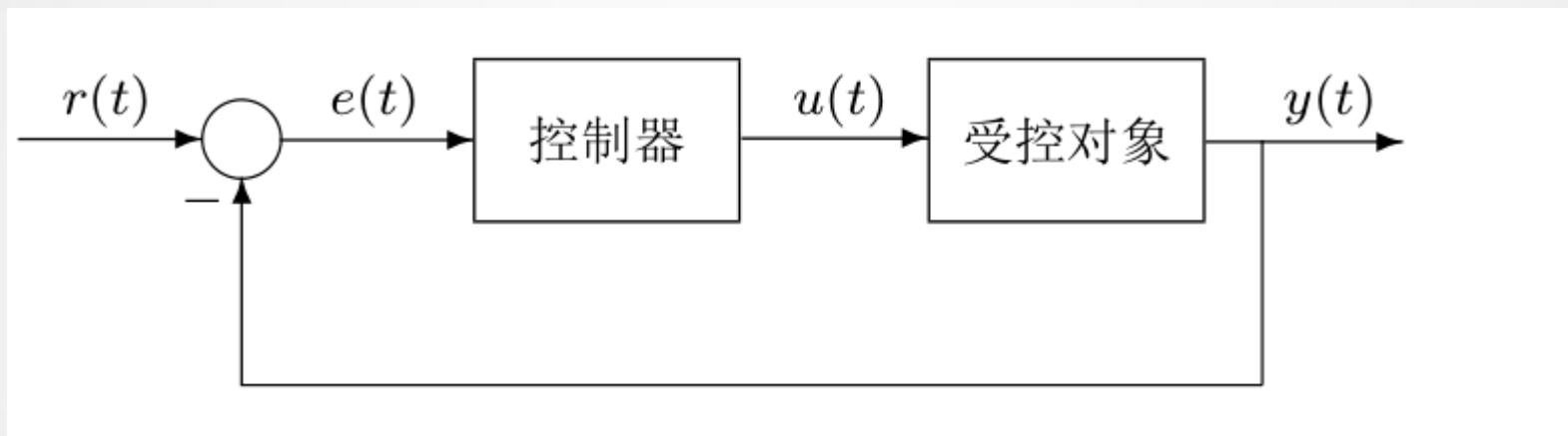


主讲：薛定宇教授



超前滞后控制器

➤ 典型反馈控制系统结构



➤ 超前滞后校正器的数学模型

$$G_c(s) = K \frac{(\alpha T_1 s + 1)(T_2 s + 1)}{(T_1 s + 1)(\beta T_2 s + 1)}$$



控制器设计的目标

- 一般伺服控制系统的要求
 - 超调量小，调节时间短、无静态误差
- 如何实现？
 - 如何实现超调量小的目标？—— 可以选择合适的相位裕度
 - 如何实现响应的快速性？—— 提高剪切频率的值
 - 如何实现无静态误差或小静态误差？
 - 引入积分器；增加控制器的增益



超前滞后校正器的设计方法

- 基于剪切频率 ω_c 和相位裕度 γ 的设计

$$G_c(s) = \frac{K_c(s + z_{c1})(s + z_{c2})}{(s + p_{c1})(s + p_{c2})}$$

- 其中 , $z_{c1} \leq p_{c1}$, $z_{c2} \geq p_{c2}$, K_c 为校正器的增益

- 设计规则 :

- 若 $\phi_c(\omega_c) > 0$, 需要引入超前的校正器

$$\alpha = \frac{z_{c1}}{p_{c1}} = \frac{1 - \sin \phi_c(\omega_c)}{1 + \sin \phi_c(\omega_c)}$$

$$z_{c1} = \sqrt{\alpha} \omega_c, \quad p_{c1} = \frac{z_{c1}}{\sqrt{\alpha}} = \frac{\omega_c}{\sqrt{\alpha}}, \quad K_c = \frac{\sqrt{\omega_c^2 + p_{c1}^2}}{\sqrt{\omega_c^2 + z_{c1}^2} A(\omega_c)}$$



➤ 系统静态误差系数为

$$K_1 = \lim_{s \rightarrow 0} s^v G_o(s) = \frac{b_m}{a_{n-v}} \frac{K_c z_{c1}}{p_{c1}}$$

➤ 超前滞后校正器进一步设计为

$$z_{c2} = \frac{\omega_c}{10}, \quad p_{c2} = \frac{K_1 z_{c2}}{K_v}$$

➤ 若 $\phi_c(\omega_c) < 0$, 则需要滞后校正器

$$K_1 = b_m \frac{K_c}{a_{n-v}}, \quad K_c = \frac{1}{A(\omega_c)}, \quad z_{c2} = \frac{\omega_c}{10}, \quad p_{c2} = \frac{K_1 z_{c2}}{K_v}$$



超前滞后校正器设计函数

➤ 调用格式 $G_c = \text{leadlagc}(G, \omega_c, \gamma, K_v, \text{key})$

➤ 清单

```
function Gc=leadlagc(G,Wc,Gam_c,Kv,key)
G=tf(G); [Gai,Pha]=bode(G,Wc);
Phi_c=sin((Gam_c-Pha-180)*pi/180);
den=G.den{1}; a=den(length(den):-1:1);
ii=find(abs(a)<=0); num=G.num{1}; G_n=num(end);
if length(ii)>0, a=a(ii(1)+1); else, a=a(1); end;
alpha=sqrt((1-Phi_c)/(1+Phi_c)); Zc=alpha*Wc;
Pc=Wc/alpha; Kc=sqrt((Wc*Wc+Pc*Pc)/(Wc*Wc+Zc*Zc))/Gai;
K1=G_n*Kc*alpha/a;
```



```
if nargin==4, key=1;
    if Phi_c<0, key=2; else, if K1<Kv, key=3; end, end
end
switch key
case 1, Gc=tf([1 Zc]*Kc,[1 Pc]);
case 2
    Kc=1/Gai; K1=G_n*Kc/a; Gc=tf([1 0.1*Wc],[1 K1*Gcn(2)/Kv]);
case 3
    Zc2=Wc*0.1; Pc2=K1*Zc2/Kv; Gcn=Kc*conv([1 Zc],[1,Zc2]);
    Gcd=conv([1 Pc],[1,Pc2]); Gc=tf(Gcn,Gcd);
end
```



例7-1 超前滞后校正器设计

➤ 受控对象模型 $G(s) = \frac{10(s+1)(s+0.5)}{s(s+0.1)(s+2)(s+10)(s+20)}$

➤ 选择 $\omega_c = 10 \text{ rad/sec}$, 相位裕度选择不同的值



```
>> s=tf('s'); G=4*(s+1)*(s+0.5)/s/(s+0.1)/(s+2)/(s+10)/(s+20)
wc=20; f1=figure; f2=figure;
for gam=20:10:90
    Gc=leadlagc(G,wc,gam,1000,3);
    figure(f1); step(feedback(G*Gc,1),1); hold on
    figure(f2); bode(Gc*G); hold on;
end
```




控制器设计结果

- 选择 $\omega_c = 20\text{rad/s}$, $\gamma = 60^\circ$



```
>> Gc1=zpk(leadlagc(G,20,60,1000,3))  
      Gc2=zpk(leadlagc(G,20,60,1000,1))  
      step(feedback(G*Gc1,1),feedback(G*Gc2,1),':',1)
```

- 超前滞后校正器

$$G_{c_1}(s) = \frac{27283.5668(s + 2.326)(s + 2)}{(s + 172)(s + 0.3173)}$$

- 超前校正器

$$G_{c_2}(s) = \frac{27283.5668(s + 2.326)}{(s + 172)}$$



不同剪切频率的设计

- 选择相位裕度为 60，不同剪切频率



```
>> gam=60; f1=figure; f2=figure;  
    for wc=5:5:30  
        Gc=leadlagc(G,wc,gam,1000,3); [a,b,c,d]=margin(Gc*G);  
        figure(f1); step(feedback(G*Gc,1),3);  
        figure(f2); bode(Gc*G); hold on;  
    end
```

- 很多组合由于要求过高，达不到
 - 设计之后应该检验



控制信号曲线绘制

➤ 设计时未考虑控制信号大小

➤ $\omega_c = 30\text{rad/s}$

 `>> Gc=leadlagc(G,30,60,1000,3);
y=step(feedback(Gc,G)); max(y)`

➤ $\gamma = 60^\circ, \omega_c = 100\text{rad/s}$

 `>> gam=60; wc=100; Gc=leadlagc(G,wc,gam,1000,3);
step(feedback(G*Gc,1),0.5);`

➤ $\omega_c = 1000\text{rad/s}$

 `>> Gc1=leadlagc(G,10*wc,gam,1000,3);
bode(Gc*G,Gc1*G)`



超前滞后校正器设计小结

- 串联控制的一般结构
- 超前、滞后、超前滞后校正器简介
- 超前滞后校正器的设计
 - 给出期望剪切频率和相位裕度
 - 利用MATLAB函数直接设计 `leadlagc()`
 - 设计出的控制器应该检验一下，是不是预期的指标能够达到，如果不能达到应该修正预期的指标重新设计
 - 后面将介绍其他的设计方法



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第七章 控制器设计的经典方法

状态空间设计方法 (上)

State Space Design Methods (I)



主讲：薛定宇教授



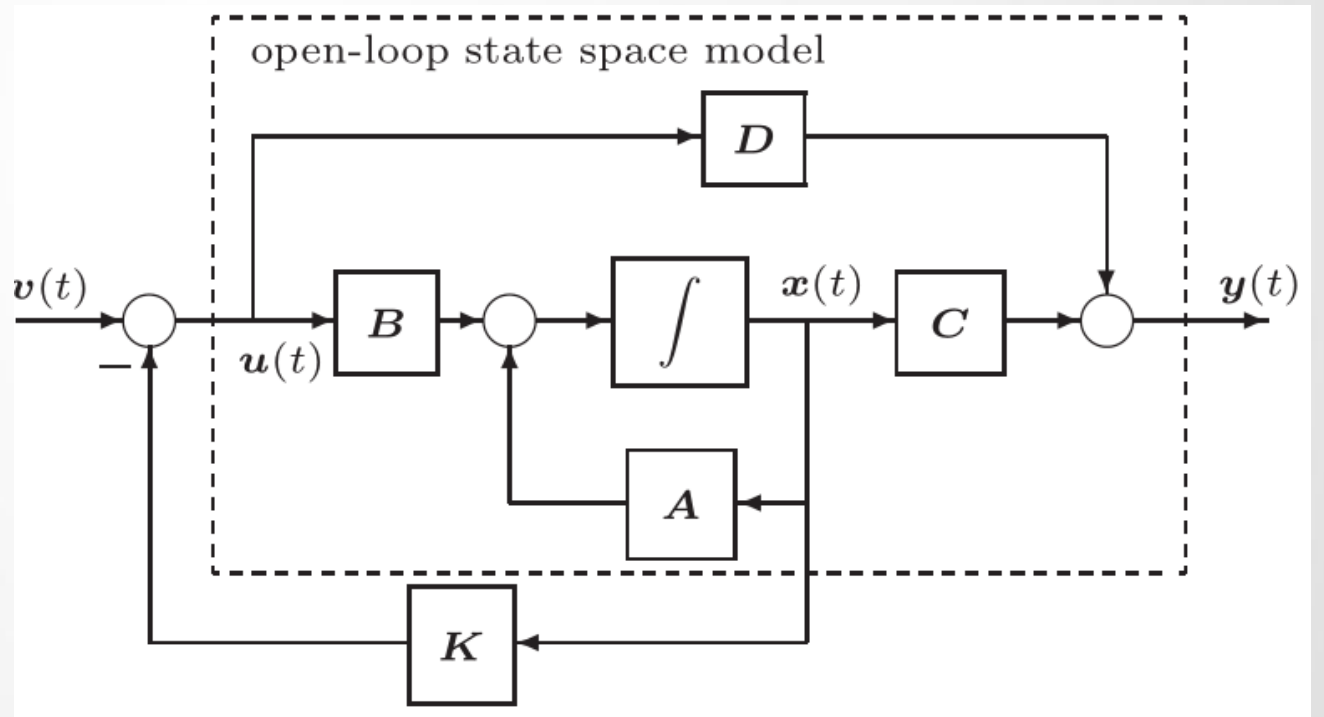
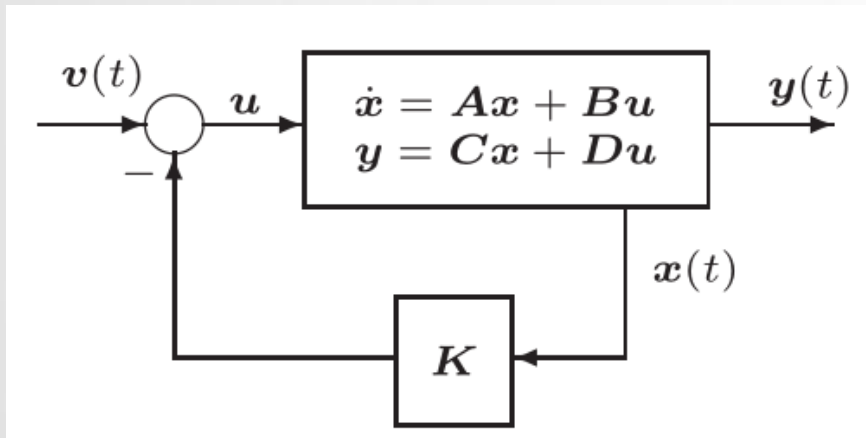
基于状态空间模型的控制器 设计方法

- 现代控制理论——状态空间方法
- 本节主要内容
 - 状态反馈系统内部和外部结构
 - 线性二次型最优调节器设计
 - 极点配置控制器设计
 - 观测器及观测器设计
 - 基于观测器的调节器与控制器设计



状态反馈控制结构

- 状态反馈的外部模型与内部模型
- 状态方程 $u(t) = v(t) - Kx(t)$





状态反馈理论基础

➤ 状态反馈与输入信号

➤ 状态反馈的闭环模型 $u(t) = v(t) - \mathbf{K}x(t)$

$$\begin{cases} \dot{x}(t) = (\mathbf{A} - \mathbf{BK})x(t) + \mathbf{B}v(t) \\ y(t) = (\mathbf{C} - \mathbf{DK})x(t) + \mathbf{D}v(t) \end{cases}$$

➤ 如果系统完全可控，则可以将闭环模型 $(\mathbf{A} - \mathbf{BK})$ 的极点配置到任意指定的位置

➤ 当然需要满足：实数或共轭复数



线性二次型指标最优调节器

➤ 受控对象——状态方程模型

$$\begin{cases} \dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) \\ \mathbf{y}(t) = \mathbf{C}\mathbf{x}(t) + \mathbf{D}\mathbf{u}(t) \end{cases}$$

➤ 设计目标 $\mathbf{u}(t)$

➤ 找出最优输入

➤ 使得目标函数最小化

$$J = \frac{1}{2} \mathbf{x}^T(t_f) \mathbf{S} \mathbf{x}(t_f) + \frac{1}{2} \int_{t_0}^{t_f} \left[\mathbf{x}^T(t) \mathbf{Q}(t) \mathbf{x}(t) + \mathbf{u}^T(t) \mathbf{R}(t) \mathbf{u}(t) \right] dt$$



线性二次型最优调节器

➤ 最优控制信号 $u^*(t) = -R^{-1}B^T P(t)x(t)$

➤ Riccati 微分方程 $P(t_f) = S$

$$\dot{P}(t) = -P(t)A - A^T P(t) + P(t)BR^{-1}B^T P(t) - Q$$

➤ 微分方程不能求解，简化为代数方程

$$PA + A^T P - PBR^{-1}B^T P + Q = 0$$

➤ MATLAB求解 $K = -R^{-1}B^T P$

$$[K, P] = \text{lqr}(A, B, Q, R)$$



离散系统的二次型最优调节器

➤ 二次型性能指标

$$J = \frac{1}{2} \sum_{k=0}^N \left[\mathbf{x}^T(k) \mathbf{Q} \mathbf{x}(k) + \mathbf{u}^T(k) \mathbf{R} \mathbf{u}(k) \right]$$

➤ 离散Riccati代数方程

$$\mathbf{S} = \mathbf{F}^T \left[\mathbf{S} - \mathbf{S} \mathbf{G} \mathbf{R}^{-1} \mathbf{G}^T \mathbf{S} \right] \mathbf{F} + \mathbf{Q}$$

➤ 控制律 $\mathbf{K} = \left[\mathbf{R} + \mathbf{G}^T \mathbf{S} \mathbf{G} \right]^{-1} \mathbf{B}^T \mathbf{S} \mathbf{F}$

➤ MATLAB求解 $[\mathbf{K}, \mathbf{S}] = \text{dlqr}(\mathbf{F}, \mathbf{G}, \mathbf{Q}, \mathbf{R})$



例7-2 状态方程模型

➤ 状态方程受控对象

$$\mathbf{A} = \begin{bmatrix} 2 & 0 & 4 & 1 & 2 \\ 1 & -2 & -4 & 0 & 1 \\ 1 & 4 & 3 & 0 & 2 \\ 2 & -2 & 2 & 3 & 3 \\ 1 & 4 & 6 & 2 & 1 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$
$$\mathbf{C} = [1, 0, 0, 0, 0]$$

➤ 加权矩阵 $\mathbf{Q} = \text{diag}(1000, 0, 1000, 500, 500)$, $\mathbf{R} = \mathbf{I}_2$

➤ 最优调节器设计



```
>> A=[2,0,4,1,2; 1,-2,-4,0,1; 1,4,3,0,2; 2,-2,2,3,3; 1,4,6,2,1];  
    B=[1,2; 0,1; 0,0; 0,0; 0,0]; C=[1,0,0,0,0];  
    Q=diag([1000 0 1000 500 500]); R=eye(2); [K,S]=lqr(A,B,Q,R)
```



```
>> G=ss(A-B*K,B,C,0); step(G)
```




国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第七章 控制器设计的经典方法

状态空间设计方法 (中)

State Space Design Methods (II)



主讲：薛定宇教授



极点配置控制器设计

➤ 受控对象——状态方程

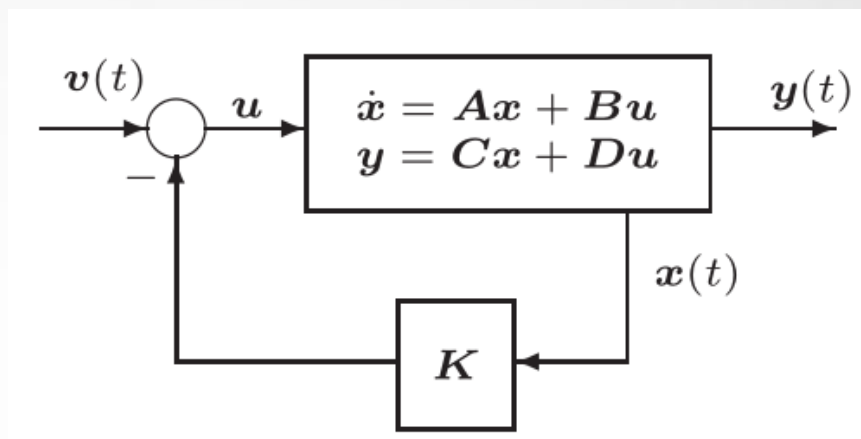
$$\begin{cases} \dot{x}(t) = Ax(t) + Bu(t) \\ y(t) = Cx(t) + Du(t) \end{cases}$$

➤ 控制信号 $u(t) = v(t) - Kx(t)$

➤ 闭环状态方程模型

$$\begin{cases} \dot{x}(t) = (A - BK)x(t) + Bv(t) \\ y(t) = (C - DK)x(t) + Dv(t) \end{cases}$$

➤ 指定闭环极点 p , 如何设计 K ?





极点配置算法

- 假设闭环系统期望的极点位置为 $\mu_i, i = 1, 2, \dots, n$
则闭环系统的特征方程 $\alpha(s)$ 可以表示成

$$\alpha(s) = \prod_{i=1}^n (s - \mu_i) = s^n + \alpha_1 s^{n-1} + \alpha_2 s^{n-2} + \dots + \alpha_{n-1} s + \alpha_n$$

- 不同的极点配置算法

➤ Bass-Gura 算法 $K = \text{bass_pp}(A, B, p)$

➤ Ackermann 算法 $K = \text{acker}(A, B, p)$

➤ 鲁棒配置算法 $K = \text{place}(A, B, p)$



Bass-Gura 算法

➤ 开环特征方程

$$a(s) = \det(s\mathbf{I} - \mathbf{A}) = s^n + a_1s^{n-1} + a_2s^{n-2} + \cdots + a_{n-1}s + a_n$$

➤ 反馈向量

$$\gamma^T = [(a_n - \alpha_n), \cdots, (a_1 - \alpha_1)], \quad \mathbf{T}_c = [\mathbf{B}, \mathbf{AB}, \cdots, \mathbf{A}^{n-1}\mathbf{B}]$$

$$\mathbf{\Gamma} = \begin{bmatrix} a_{n-1} & a_{n-2} & \cdots & a_1 & 1 \\ a_{n-2} & a_{n-3} & \cdots & 1 & \\ \vdots & \vdots & \ddots & & \\ a_1 & 1 & & & \\ 1 & & & & \end{bmatrix}$$

$$\mathbf{K} = \gamma^T \mathbf{\Gamma}^{-1} \mathbf{T}_c^{-1}$$



Bass-Gura算法的MATLAB实现

➤ Bass-Gura算法编程

```
function K=bass_pp(A,B,p)
a1=poly(p); a=poly(A); L=hankel(a(end-1:-1:1)); C=ctrb(A,B);
K=(a1(end:-1:2)-a(end:-1:2))*inv(L)*inv(C);
```

➤ Ackermann 算法

$$K = -[0, 0, \dots, 0, 1] T_c^{-1} \alpha(A)$$

➤ 函数的区别

- place() 函数能处理多变量系统
- 另外两种方法可以处理多重极点配置问题



例7-3 极点配置

➤ 状态方程

$$\dot{\mathbf{x}}(t) = \begin{bmatrix} 0 & 2 & 0 & 0 & -2 & 0 \\ 1 & 0 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 & 0 \\ 2 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 1 & 0 \end{bmatrix} \mathbf{x}(t) + \begin{bmatrix} 1 & 2 \\ 0 & 0 \\ 0 & 1 \\ 0 & -1 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} \mathbf{u}(t)$$

➤ 期望极点位置 $-1, -2, -3, -4, -1 \pm j$



```
>> A=[0,2,0,0,-2,0; 1,0,0,0,0,-1; 0,1,0,0,0,0;  
      0,0,0,3,0,0; 2,0,0,1,0,0; 0,0,-1,0,1,0];  
B=[1,2; 0,0; 0,1; 0,-1; 0,1; 0,0];  
p=[-1 -2 -3 -4 -1+1i -1-1i];  
K=place(A,B,p), p1=eig(A-B*K)
```




例7-4 极点配置失败

➤ 状态方程

$$\boldsymbol{x}[(k+1)T] = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 5 & 0 \end{bmatrix} \boldsymbol{x}(kT) + \begin{bmatrix} 0 & 1 \\ 0 & -1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \boldsymbol{u}(kT)$$

➤ 期望极点 $-0.1, -0.2, -0.5 \pm 0.2j$

➤ 控制器设计 (不能配置)



```
>> A=[0 1 0 0 ; 0 0 -1 0; 0 0 0 1; 0 0 5 0];  
    B=[0 1 ; 0 -1; 0 0 ; 0 0];  
    p=[-0.1; -0.2; -0.5+0.2i; -0.5-0.2i]; K=place(A,B,p)
```

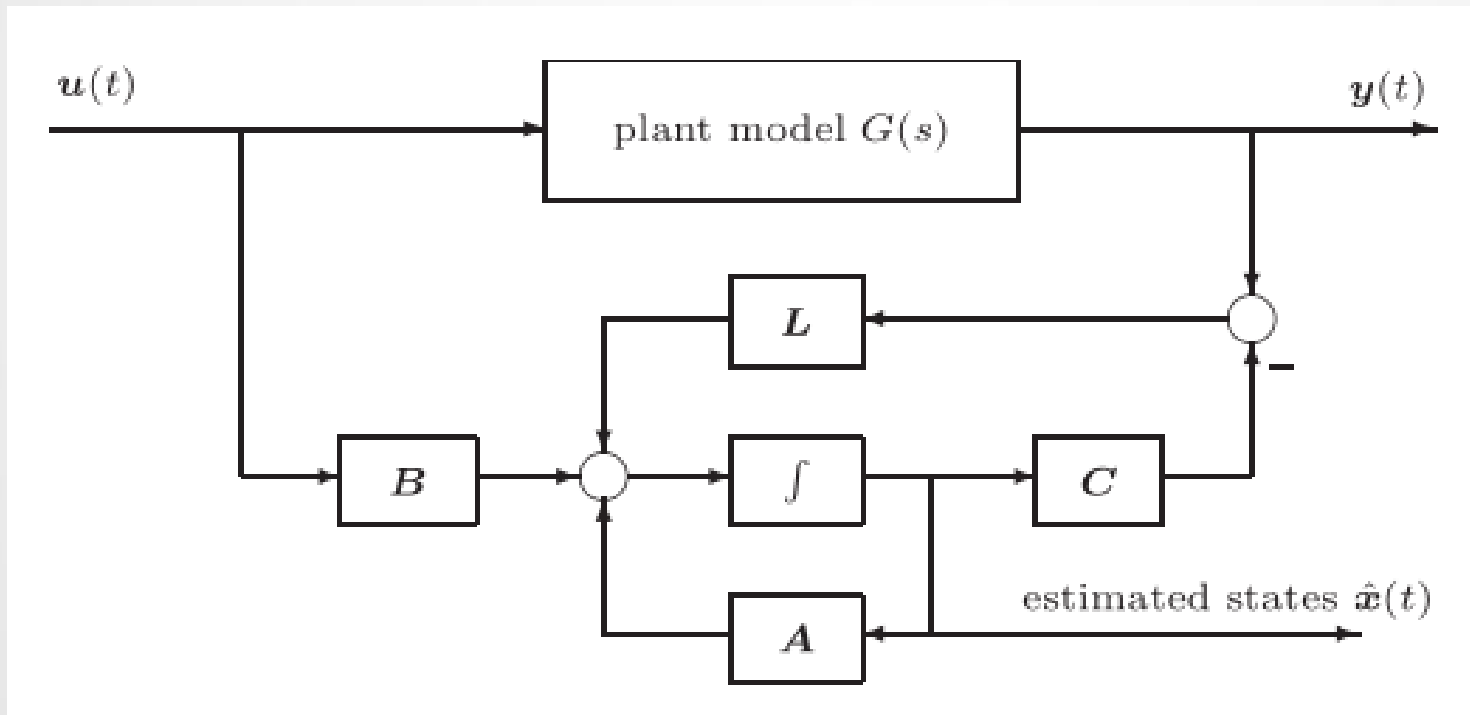


```
>> rank(ctrb(A,B))
```



观测器的框图

- 如果状态不能直接测出，则需要观测器重建状态





观测器稳定性和收敛性

➤ 观测器数学模型

$$\begin{aligned}\dot{\hat{x}}(t) &= A\hat{x}(t) + Bu(t) - L(C\hat{x}(t) + Du(t) - y(t)) \\ &= (A - LC)\hat{x}(t) + (B - LD)u(t) + Ly(t)\end{aligned}$$

➤ 观测器收敛性

$$\begin{aligned}\dot{\hat{x}}(t) - \dot{x}(t) &= (A - LC)\hat{x}(t) + (B - LD)u(t) + Ly(t) - Ax(t) - Bu(t) \\ &= (A - LC)[\hat{x}(t) - x(t)]\end{aligned}$$

➤ 方程解析解 $\hat{x}(t) - x(t) = e^{(A-LC)(t-t_0)}[\hat{x}(t_0) - x(t_0)]$

➤ $(A - LC)$ 稳定性 $\lim_{t \rightarrow \infty} [\hat{x}(t) - x(t)] = 0$



观测器仿真与设计

➤ MATLAB程序

```
function [xh,x,t]=simobsv(G,L)
[y,t,x]=step(G); G=ss(G); A=G.a; B=G.b; C=G.c; D=G.d;
[y1,xh1]=step((A-L*C),(B-L*D),C,D,1,t);
[y2,xh2]=lsim((A-L*C),L,C,D,y,t); xh=xh1+xh2;
```

➤ 调用格式

$$[\hat{x}, x, t] = \text{simobsv}(G, L)$$



例7-5 状态观测与仿真

➤ 原始状态方程模型

$$\dot{\mathbf{x}}(t) = \begin{bmatrix} 0 & 2 & 0 & 0 \\ 0 & -0.1 & 8 & 0 \\ 0 & 0 & -10 & 16 \\ 0 & 0 & 0 & -20 \end{bmatrix} \mathbf{x}(t) + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0.3953 \end{bmatrix} u(t)$$

➤ 输出方程

$$y(t) = 0.09882x_1(t) + 0.1976x_2(t)$$

➤ 如何设计观测器重构系统的状态？



观测器的仿真

➤ 状态观测器设计

➤ 期望极点 -1,-2,-3,-4



```
>> A=[0,2,0,0; 0,-0.1,8,0; 0,0,-10,16; 0,0,0,-20];  
B=[0;0;0;0.3953]; C=[0.09882,0.1976,0,0]; D=0;  
P=[-1; -2; -3; -4];  
L=place(A',C',P)'; [xh,x,t]=simobsv(ss(A,B,C,D),L);  
plot(t,x,t,xh,':'); axis([0,15,-0.5,4])
```

➤ 期望极点： -10



```
>> P=[-10;-10;-10;-10]; L=acker(A',C',P)'; L'  
[xh,x,t]=simobsv(ss(A,B,C,D),L);  
plot(t,x,t,xh,':'); axis([0,30,-0.5,4])
```



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第七章 控制器设计的经典方法

状态空间设计方法 (下)

State Space Design Methods (III)

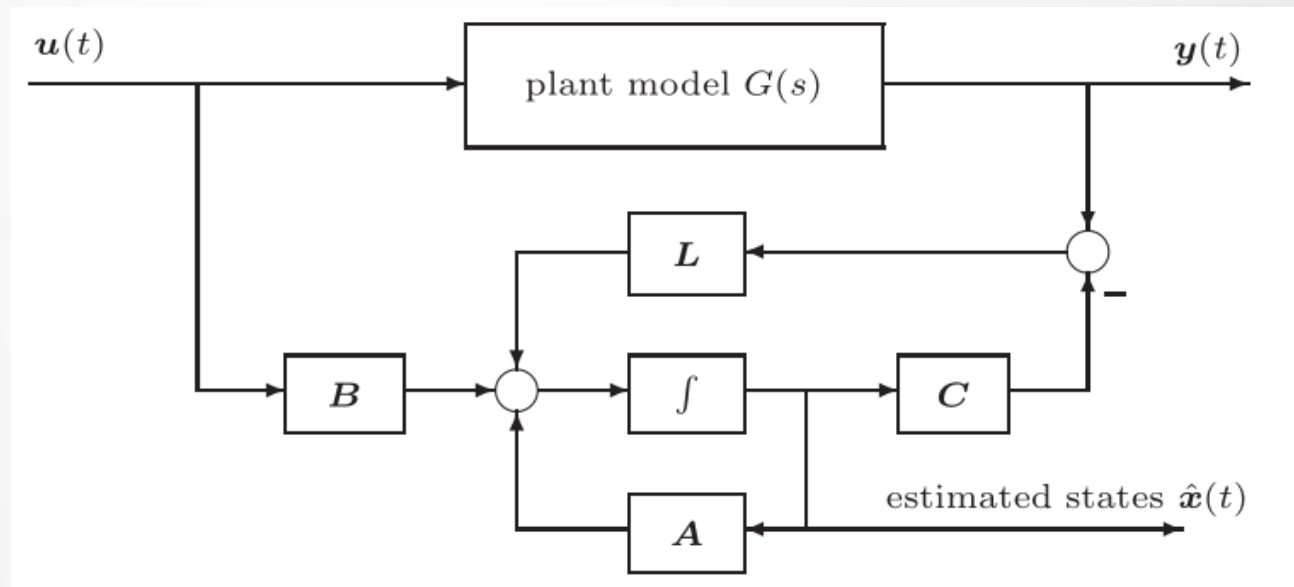


主讲：薛定宇教授



观测器模型回顾

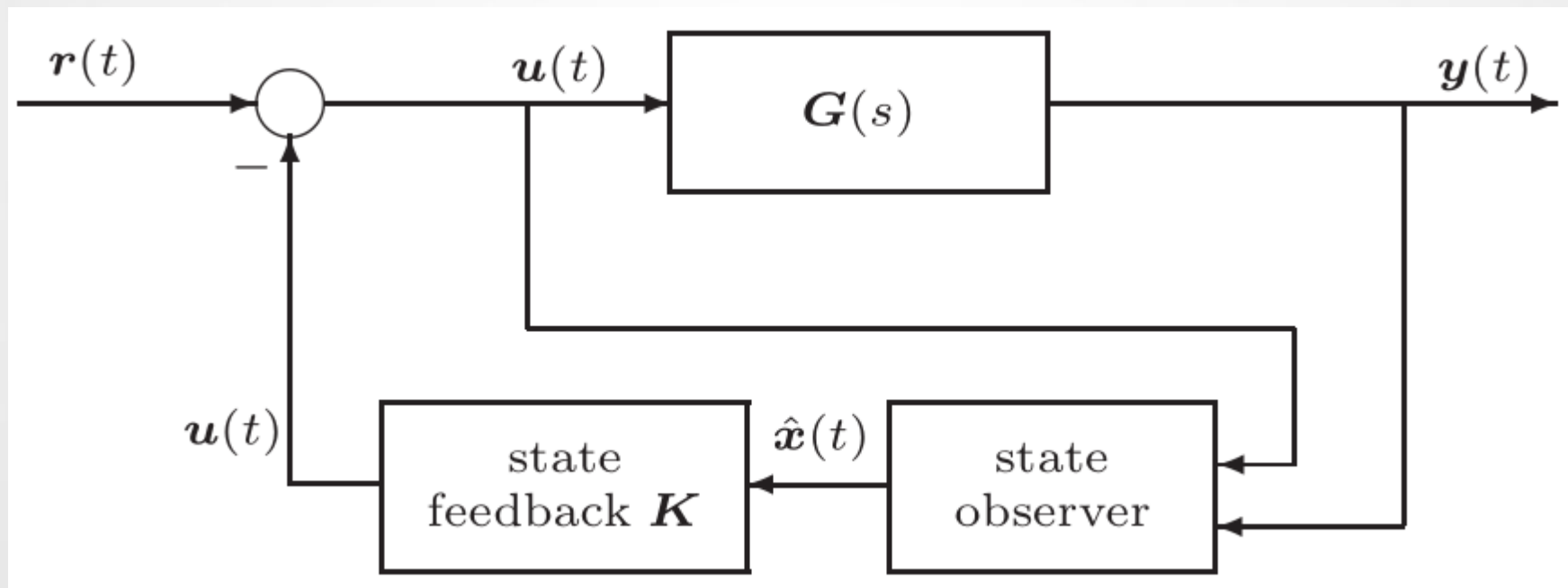
- 观测器模型
 - 由输入与输出信号重建系统的状态
 - 需要设计 L 矩阵





基于观测器的调节器与控制器

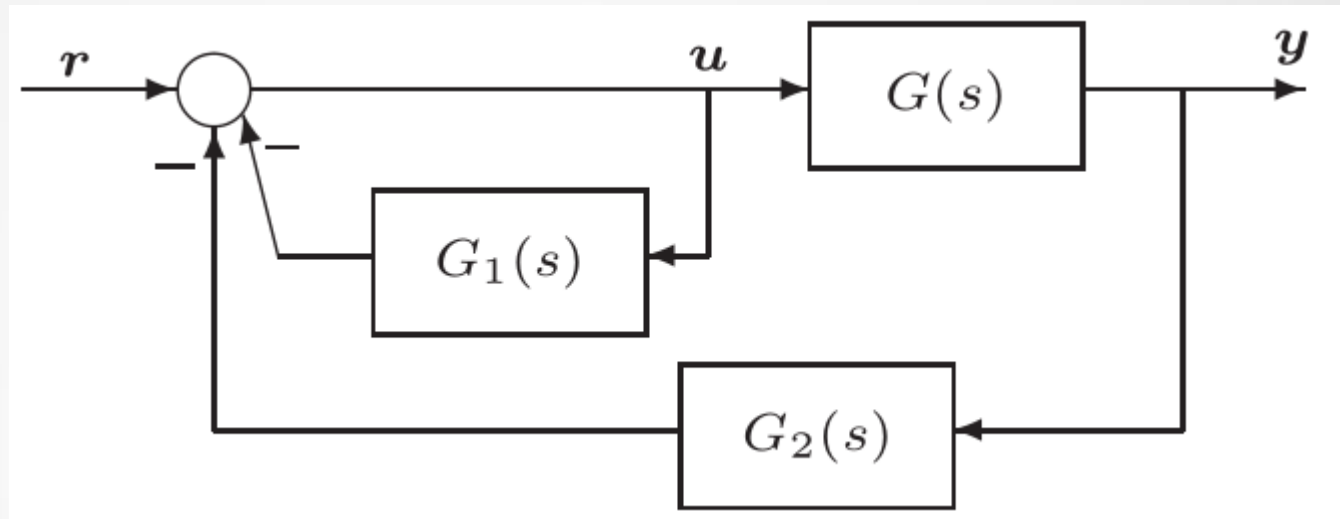
- 带有观测器的控制系统框图
- 状态未知，需要观测器重建





基于状态观测器的调节器

➤ 模型等效变换



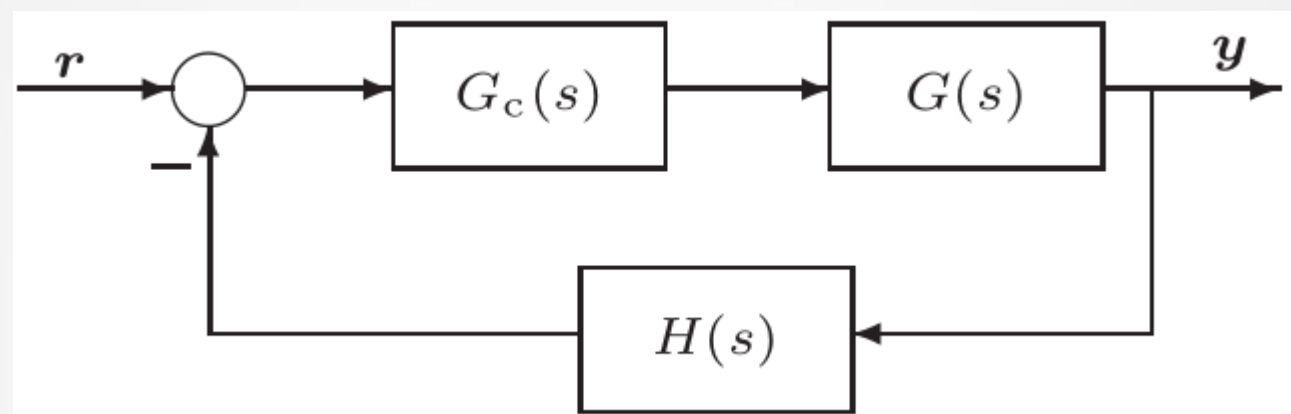
$$\begin{aligned} \text{➤ } G_1(s) \quad & \begin{cases} \dot{\hat{x}}_1(t) = (A - LC)\hat{x}_1(t) + (B - LD)u(t) \\ y_1(t) = K\hat{x}_1(t) \end{cases} \end{aligned}$$

$$\begin{aligned} \text{➤ } G_2(s) \quad & \begin{cases} \dot{\hat{x}}_2(t) = (A - LC)\hat{x}_2(t) + Ly(t) \\ y_2(t) = K\hat{x}_2(t) \end{cases} \end{aligned}$$



结构图化成典型反馈系统模型

➤ 等效框图



➤ 等效关系

➤ $G_c(s)$

$$G_c(s) = I - K(sI - A + BK + LC - LDK)^{-1}B$$

➤ 状态方程

$$\begin{cases} \dot{x}(t) = (A - BK - LC + LDK)x(t) + Bu(t) \\ y(t) = -Kx(t) + u(t) \end{cases}$$



基于观测器的控制器等效变换

➤ 等效关系

$$\begin{cases} \dot{x}(t) = (A - BK - LC + LDK)x(t) + Bu(t) \\ y(t) = -Kx(t) + u(t) \end{cases}$$

$$\begin{cases} \dot{\hat{x}}_2(t) = (A - LC)\hat{x}_2(t) + Ly(t) \\ y_2(t) = K\hat{x}_2(t) \end{cases}$$

➤ 根据等效关系可以编写出MATLAB函数

```
function [Gc,H]=obsvsf(G,K,L)
H=ss(G.a-L*G.c,L,K,0);
Gc=ss(G.a-G.b*K-L*G.c+L*G.d*K,G.b,-K,1);
```

➤ 若参考输入为0（调节问题） $G_c = \text{reg}(G, K, L)$



例7-6 系统的等效实现

➤ 受控对象模型

$$\dot{\mathbf{x}}(t) = \begin{bmatrix} 0 & 2 & 0 & 0 \\ 0 & -0.1 & 8 & 0 \\ 0 & 0 & -10 & 16 \\ 0 & 0 & 0 & -20 \end{bmatrix} \mathbf{x}(t) + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0.3953 \end{bmatrix} u(t)$$

$$y(t) = 0.09882x_1(t) + 0.1976x_2(t)$$

➤ 二次型性能指标的加权矩阵

$$R = 1, \quad Q = \text{diag}(0.01, 0.01, 2, 3)$$



状态反馈控制系统的仿真

➤ 直接设计控制器

```
 >> A=[0,2,0,0; 0,-0.1,8,0; 0,0,-10,16; 0,0,0,-20];  
      B=[0;0;0;0.3953]; C=[0.09882,0.1976,0,0]; D=0;  
      Q=diag([0.01,0.01,2,3]); R=1;  
      K=lqr(A,B,Q,R), step(ss(A-B*K,B,C,D))
```

➤ 假设状态不可测，需要构造观测器

```
 >> P=[-5;-5;-5;-5]; G=ss(A,B,C,D);  
      L=acker(A',C',P)'; [Gc,H]=obsvsvf(G,K,L);  
      step(ss(A-B*K,B,C,D),feedback(G*Gc,H))
```

➤ 最小实现

```
 >> zpk(minreal(feedback(G*Gc,H)))  
      zpk(minreal(ss(A-B*K,B,C,D)))
```



状态空间设计方法小结

- 状态反馈系统的结构
- 状态空间设计方法
 - 线性二次型性能指标的最优设计 `lqr()`、`dlqr()`
 - 极点配置的：`bass_pp()`、`place()`、`acker()`函数
- 系统状态不可测时，引入观测器重建状态
 - 状态观测器仿真：`simobsv()` 函数
 - 基于观测器的控制器：`obsvf()` 函数
 - 基于观测器的调节器：`reg()` 函数



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第七章 控制器设计的经典方法

最优控制器设计(上)

Design of Optimum Controllers (I)



主讲：薛定宇教授



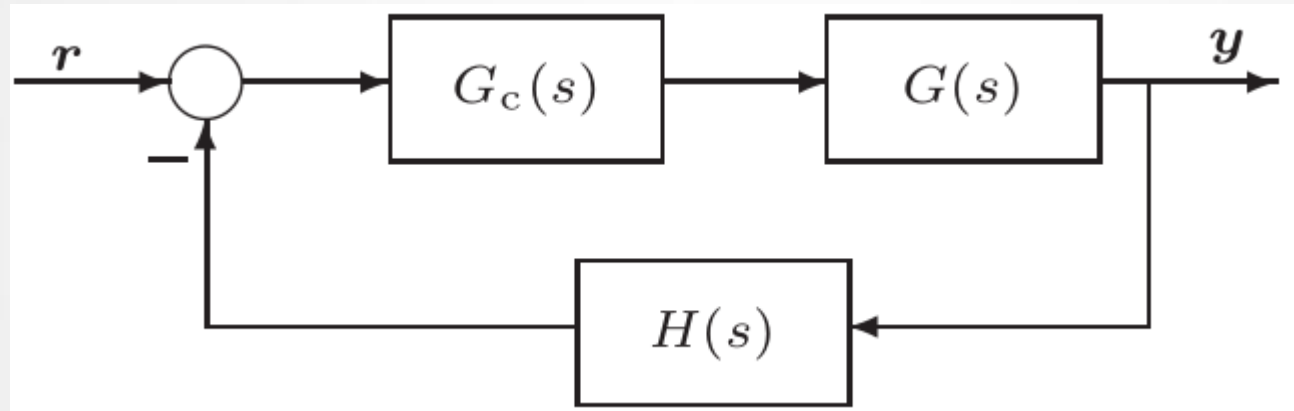
最优控制器设计

- LQR等最优控制存在的问题
 - Q, R 选择认为因素很大，缺乏客观描述
 - LQR 类控制的目标在于解析求解，如果某个问题不可求解，则试图简化问题。LQR问题已经远远偏离实际的要求
- 本节内容
 - 伺服控制选择什么样的性能指标合适
 - 如何利用数值最优化技术设计最优控制器
 - 最优控制器设计界面——OCD



伺服控制系统的要求

➤ 伺服控制框图



- 假如 $H(s) = 1$ ，则控制目标是让输出尽快跟踪输入
- 超调量小，调节时间短，跟踪的总体误差小
- 系统中如果有非线性，传统设计方法难以应用



最优控制的概念

➤ 所谓“最优控制”，就是在一定的具体条件下，要完成某个控制任务，使得选定指标最小或最大的控制，这里所谓指标就是目标函数。

➤ 有意义的性能指标

➤ 动态误差信号的积分指标

$$J_{\text{ISE}} = \int_0^{\infty} e^2(t)dt, \quad J_{\text{IAE}} = \int_0^{\infty} |e(t)|dt, \quad J_{\text{ITAE}} = \int_0^{\infty} t|e(t)|dt$$

➤ 速度最快、能量最省等



例7-7 系统的最优PID控制器设计

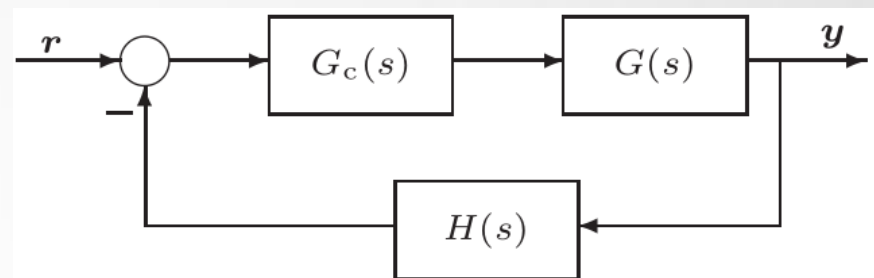
- 复杂的受控对象 $G(s) = \frac{1 + \frac{3e^{-s}}{s+1}}{s+1}$
- 误差传递函数的计算

$$E(s) = \frac{1}{1 + G(s)G_c(s)} R(s)$$

- PID控制器的描述

$$G_c(s) = K_p + K_i \frac{1}{s} + K_d s$$

$$\mathbf{x} = [K_p, K_i, K_d]$$





控制器的最优设计

➤ 目标函数的编写 $J_{ISE} = \int_0^{\infty} e^2(t)dt$

```
function y=c7fopt(x,s,G,t)
Gc=x(1)+x(2)/s+x(3)*s; E=1/(1+G*Gc);
y0=step(E,t); y=sum(y0.^2)*(t(2)-t(1));
```

➤ 最优化求解



```
>> s=tf('s'); G1=3/(s+1); G1.ioDelay=3; G=(1+G1)/(s+1);
t=0:0.02:30; x=fminunc(@c7fopt,[1,1,1],optimset,s,G,t)
```



```
>> Gc=x(1)+x(2)/s+x(3)*s;
step(feedback(G*Gc,1),30)
```



ITAE 性能指标下的最优控制器设计

- ITAE 性能指标的目标函数描述 $J_{ITAE} = \int_0^{\infty} t|e(t)|dt$

```
function y=c7fopt2(x,s,G,t)
Gc=x(1)+x(2)/s+x(3)*s; E=1/(1+G*Gc);
y0=step(E,t); y=t*abs(y0)*(t(2)-t(1));
```

- 最优控制器设计



```
>> x=fminunc(@c7fopt2,[1,1,1],optimset,s,G,t)
```



```
>> Gc=x(1)+x(2)/s+x(3)*s;
step(feedback(G*Gc,1),30)
```



哪种性能指标更合理？

- 不同性能指标得出的控制效果不同，但都是最优解
 - 哪种性能指标是最好的呢？
 - 哪种是最客观的呢？
- ISE同等处理各个时刻的误差
- ITAE对时间加权，时间 t 大，会迫使误差降下来
 - 相比之下，更适合于伺服控制
 - ISE 性能指标可以通过范数计算，可以不经过仿真
 - 现在有了强大的仿真工具，没有必要再依赖ISE指标



这里设计方法的局限性

- 受控对象和控制器必须是线性的
 - 否则不能用step函数进行仿真
 - 实际应用中，可能PID控制器输出过大，需要接饱和非线性
- 应该如何实现有意义的设计
- 充分利用MATLAB的强大功能
 - 用数值最优化语句设计控制器参数
 - 用Simulink的强大仿真功能去处理任意复杂系统的仿真



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第七章 控制器设计的经典方法

最优控制器设计(中)

Design of Optimum Controllers (II)



主讲：薛定宇教授



基于**Simulink**的最优控制器设计步骤

- 将需要设计的控制系统模型
 - 用Simulink绘制出来，并将决策变量用变量表示
 - 在Simulink模型中定义目标函数
 - 用MATLAB写出目标函数文件，用assignin()函数将Simulink中的控制器变量与目标函数建立起来联系
- 调用最优化问题求解程序直接设计控制器
 - 无约束最优化求解：fminsearch()、fminunc()
 - 有约束最优化，定义约束：fmincon()



例7-8 用基于Simulink的方法设计

➤ 受控对象模型

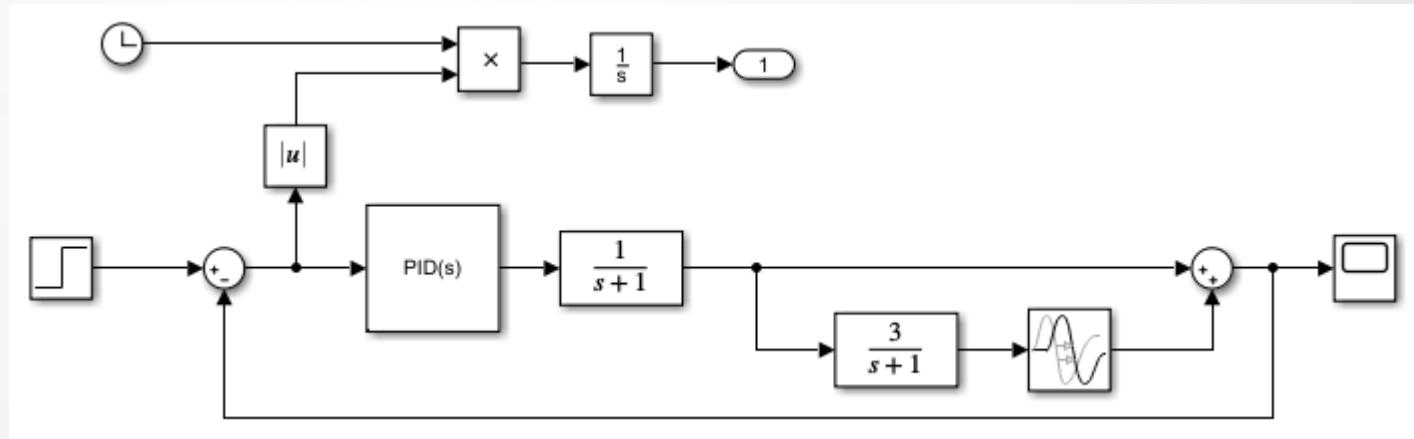
$$G(s) = \frac{1 + \frac{3e^{-s}}{s+1}}{s+1}$$

➤ 绘制Simulink框图

➤ 模型 c7mpidsys

➤ 工作空间变量

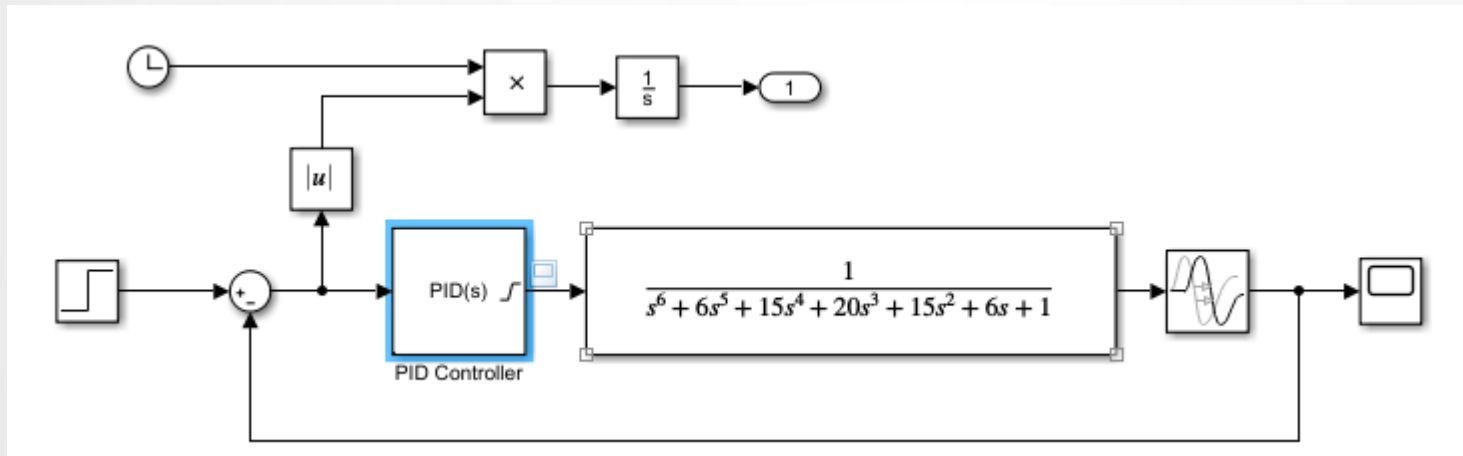
➤ Kp, Ki, Kd





例7-9 另一个控制器设计问题

- 已知受控对象模型 $G(s) = e^{-s}/(s + 1)^6$
- 如何设计最优PID控制器？





将问题转换为数值最优化问题

- 已知受控对象模型 $G(s) = e^{-s}/(s + 1)^6$
- 如何设计最优PID控制器？

```
function y=c7fopt4(x)
assignin('base','Kp',x(1));
assignin('base','Ki',x(2));
assignin('base','Kd',x(3));
[t,~,yy]=sim('c7mpidsys1',[0,30]); y=yy(end,1);
```



```
>> x=fminsearch(@c7fopt4,[1,1,1])
```

- 引入驱动饱和 $|u(t)| \leq 10$



最优控制器设计的优点

- 利用了MATLAB、Simulink各自的优势
 - MATLAB的统一框架与最优化功能
 - Simulink的强大仿真功能
 - 理论上可以处理任意复杂系统的设计
 - 受控对象、控制器都可以带有时变、非线性环节
- 同等处理非线性系统，这时其他方法不能实现的



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第七章 控制器设计的经典方法

最优控制器设计(下)

Design of Optimum Controllers (III)



主讲：薛定宇教授



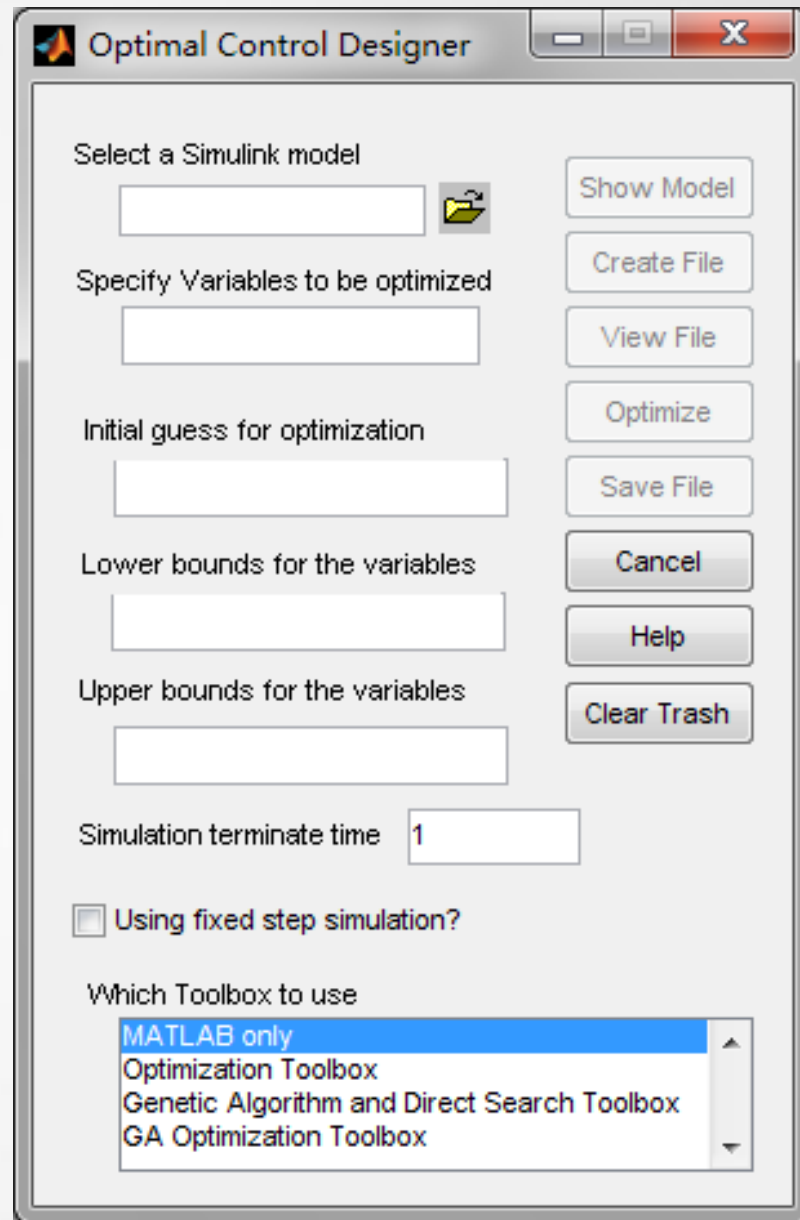
最优控制器设计界面——OCD

- OCD = Optimal Controller Designer , 2005
- 用MATLAB编写的程序界面
 - 用户由用 Simulink 画出仿真模型并定义出目标函数
 - 如 ITAE 准则、ISE准则等
 - 给 ocd 命令启动程序
 - 选择模型名、优化变量、终止时间
 - 按 Create File 按钮自动生成目标函数文件
 - 按 Optimize 即可启动最优设计过程
 - 设计时打开示波器观察优化过程



OCD程序界面

- 需要用Simulink画出控制系统
 - 实际的控制系统框图
 - 定义出误差准则
- 可以人工选择初值
- 可以指定决策变量的上下限
- 可以选择不同的优化算法
- 无须给出如何命令，即可设计出最优控制器，可以解决非线性系统问题

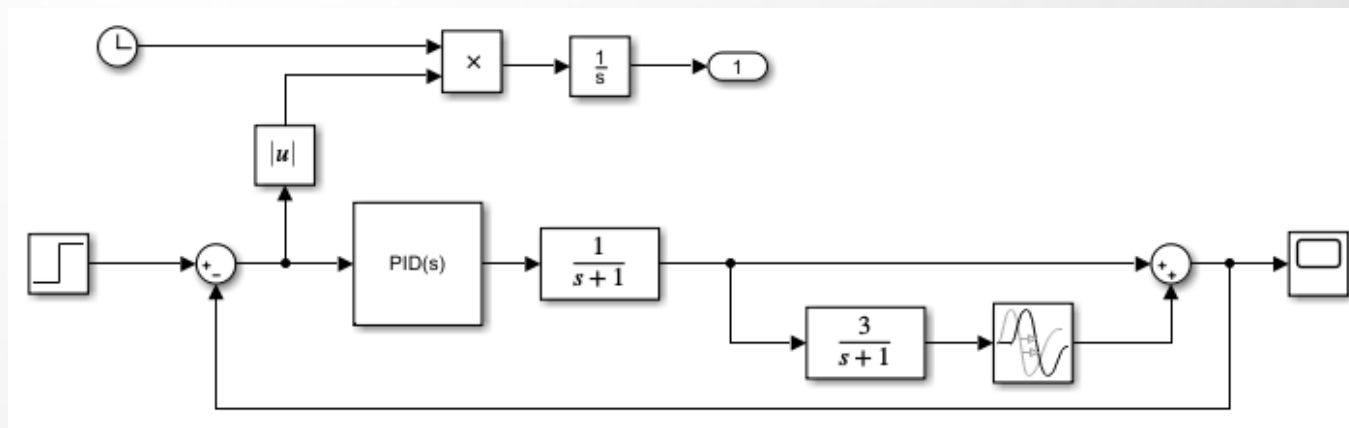




例7-10 OCD设计举例

- 受控对象
$$G(s) = \frac{1 + \frac{3e^{-s}}{s+1}}{s+1}$$
- PID控制器设计：
 - 绘制Simulink 模型：c7mpidsys.slx

- 优化参数：K_p, K_i, K_d
- 终止时间：30





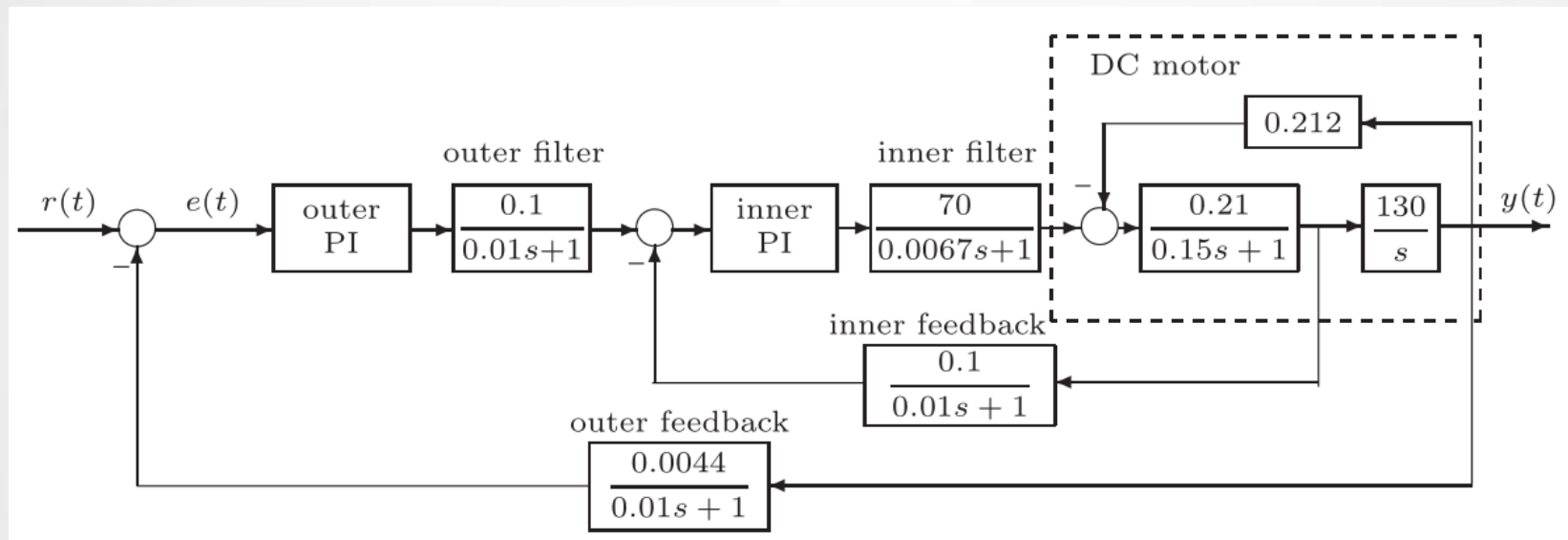
最优控制器设计

- 必要的参数
 - Simulink模型的文件名
 - 需要优化的决策变量名，用逗号分隔
 - 终止仿真时间
- 自动生成目标函数的 MATLAB文件
- 按按钮设计控制器
 - 修改目标函数，如 ISE，比较结果
 - 演示参数变化、饱和非线性引入等控制器设计的结果



例7-10 串级PI控制器设计

➤ 双闭环DC调速系统



➤ Simulink仿真模型：c7model2.mdl

➤ 优化参数：Kp1, Kp2, Ki1, Ki2

➤ 终止时间：tn=0.6



OCD的其他应用

- 从理论上说，只要能画出来误差目标函数、可以指定决策变量的问题就可以用OCD程序界面直接求解
- 其他应用
 - 模型降阶
 - 如果给出原始模型，可以用OCD逼近其模型参数
 - 后续内容可以尝试使用OCD，如模型参考自适应控制系统
 - 对于PID控制器设计，建议使用后面更专门的optimpid程序



例7-11 最优降阶的例子

➤ 原始模型 $G(s) = 1/(s+1)^6$

➤ FOPDT

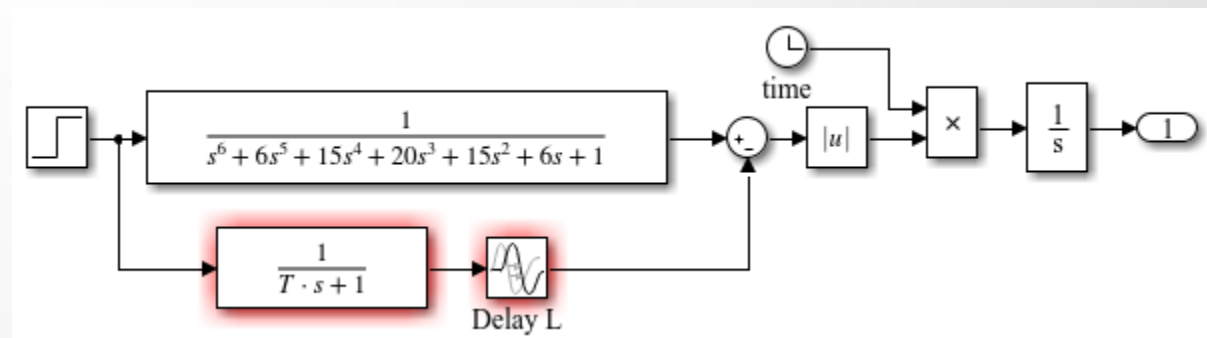
$$G_1(s) = \frac{k}{Ts + 1} e^{-Ls}$$

➤ 由静态误差相同可以得出 $k = 1$

➤ Simulink模型 : c7mmr.mdl

➤ 优化参数 : L, T

➤ 终止时间 : 10

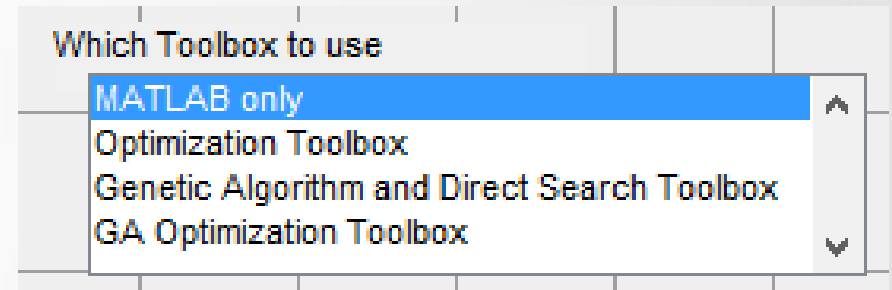




OCD程序的编程简介

- 可以由 `guide ocd` 命令打开编辑界面
- 开放的结构，如控制器设计

```
switch kTool
    case 1
        ctrl_pars=fminsearch(str,x0,options)
    case 2
        ctrl_pars=fminunc(str,x0,options)
    case 3
        eval(['ctrl_pars=ga(@' str ', ' int2str(length(x0)) ')'])
    case 4
        ctrl_pars=gaopt([zeros(size(x0(:))),100*ones(size(x0(:)))],str)
end
```





最优控制器设计小结

- 探讨了目标函数的选择问题
 - 演示了ITAE类指标比ISE指标更适合伺服控制
- 结合数值最优化技术和Simulink建模仿真技术
- 给出了对任意复杂系统的最优控制器设计方法
 - 使用 `assignin()`、`fminsearch()` 等函数
- 演示了作者开发的 OCD 图形用户界面
 - 用户需要提供 Simulink 框图
 - 指出待优化变量和终止时间等参数
 - 按动相关按钮即可“可视”优化过程



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第七章 控制器设计的经典方法

多变量频域设计 (上)

Frequency Domain Design of Multivariable Systems (I)



主讲：薛定宇教授



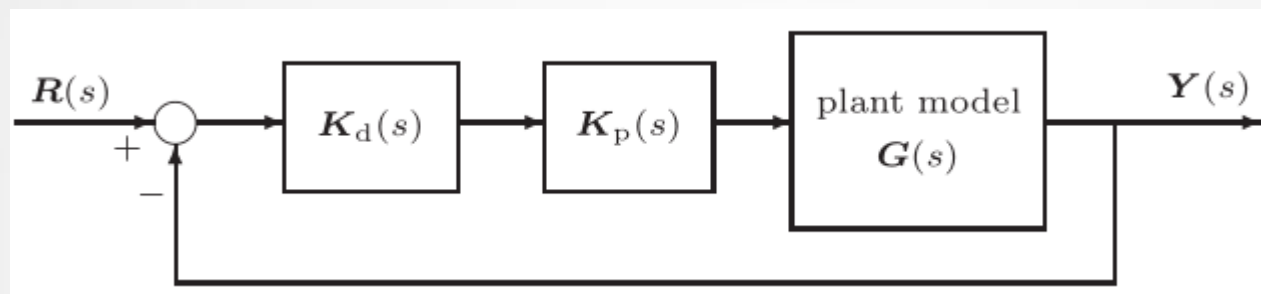
多变量系统的频域设计方法

- 多变量系统比单变量系统设计难很多
 - 主要难点：系统不同输入输出对之间的耦合，难点在如何解耦？
 - 解耦后可以单独回路设计，沿用单变量的设计方式，给每回路单独设计控制器，不影响其他
- 本节主要内容
 - 对角占优化方法与伪对角化方法
 - 参数最优化方法



对角占优系统与伪对角化

➤ 典型多变量反馈系统框图



- $K_p(s)$ 预补偿矩阵，使 $G(s) K_p(s)$ 为对角占优矩阵
- $K_d(s)$ 为动态补偿矩阵
- $K_p(s)$ 可以用试凑方法，可以取 $K_p(s) = G^{-1}(0)$
- 还可以采用其他方法，如伪对角化方法



伪对角化方法

- 在 $j\omega_0$ 点处的逆Nyquist阵列 (INA) 为

$$\hat{g}_{ik}(j\omega_0) = \alpha_{ik} + j\beta_{ik}, \quad i, k = 1, 2, \dots, m$$

- 假定输入、输出路数相同。步骤：

- 选择一个频率点 $j\omega_0$, 求出 INA $\hat{g}_{ik}(j\omega_0)$

- 对各个 q , 构造

$$a_{il,q} = \sum_{k=1, k \neq q}^m \left[\alpha_{ik}\alpha_{lk} + \beta_{ik}\beta_{lk} \right], \quad i, l = 1, 2, \dots, m$$

- 求 A_q 矩阵的特征向量 , 最小特征值的向量 k_q

- 由各个 q 对应的最小特征向量构造补偿矩阵



多点伪对角化

➤ 上述算法的 $j\omega_0$ 可以扩展到一系列频率

➤ 频率点 $\omega_1, \omega_2, \dots, \omega_N$

➤ 对第 r 频率点加权 ψ_r ，构造

$$A_{il,q} = \sum_{r=1}^N \psi_r \left[\sum_{k=1 \text{ and } k \neq q}^m (\alpha_{ik,r} \alpha_{lk,r} + \beta_{ik,r} \beta_{lk,r}) \right]$$

➤ 基本思想：在某几个频率点实现对角化

➤ 编写MATLAB函数生成补偿矩阵

➤ 调用格式 $K_p = \text{psuediag}(G, R)$



伪对角化算法的实现

➤ 调用格式 $K_p = \text{psuediag}(G, R)$

```
function Kp=psuediag(G1,R)
A=real(G1); B=imag(G1); [n,m]=size(G1); N=n/m; Kp=[];
if nargin==1, R=ones(N,1); end
for q=1:m, L=[1:q-1, q+1:m];
    for i=1:m, for l=1:m, a=0;
        for r=1:N, k=(r-1)*m;
            a=a+R(r)*sum(A(k+i,L).*A(k+1,L)+B(k+i,L).*B(k+1,L));
        end,
        Ap(i,l)=a;
    end, end
    [x,d]=eig(Ap); [xm,ii]=min(diag(d)); Kp=[Kp; x(:, ii)'];
end
```



例7-15 多变量问题伪对角化

➤ 蒸汽锅炉温度控制模型

$$G(s) = \begin{bmatrix} 1/(1+4s) & 0.7/(1+5s) & 0.3/(1+5s) & 0.2/(1+5s) \\ 0.6/(1+5s) & 1/(1+4s) & 0.4/(1+5s) & 0.35/(1+5s) \\ 0.35/(1+5s) & 0.4/(1+5s) & 1/(1+4s) & 0.6/(1+5s) \\ 0.2/(1+5s) & 0.3/(1+5s) & 0.7/(1+5s) & 1/(1+4s) \end{bmatrix}$$

➤ 原系统



```
>> s=tf('s'); w=logspace(-1,0);  
G=[1/(1+4*s), 0.7/(1+5*s), 0.3/(1+5*s), 0.2/(1+5*s);  
    0.6/(1+5*s), 1/(1+4*s), 0.4/(1+5*s), 0.35/(1+5*s);  
    0.35/(1+5*s), 0.4/(1+5*s), 1/(1+4*s), 0.6/(1+5*s);  
    0.2/(1+5*s), 0.3/(1+5*s), 0.7/(1+5*s), 1/(1+4*s)];  
H=mfrd(G,w); gershgorin(H)
```



伪对角化

- 选定预校正矩阵 $K = G^{-1}(0)$

 `>> K=inv(mfrd(G,0)); W=mfrd(G*K,w); gershgorin(W)`

- 选频率点

 `>> v=0.9; iH=mfrd(inv(G),v); Kp=inv(pseuddiag(iH)),
V=mfrd(G*Kp,w); gershgorin(V)`

- 选一组频率点

 `>> v=logspace(-2,log10(0.4)); iH=mfrd(inv(G),v);
Kp=inv(pseuddiag(iH)), Q=mfrd(G*Kp,w);
gershgorin(Q)`



例7-16 多变量延迟系统

➤ 系统模型

$$G(s) = \begin{bmatrix} \frac{0.1134e^{-0.72s}}{1.78s^2 + 4.48s + 1} & \frac{0.924}{2.07s + 1} \\ \frac{0.3378e^{-0.3s}}{0.361s^2 + 1.09s + 1} & \frac{-0.318e^{-1.29s}}{2.93s + 1} \end{bmatrix}$$

➤ 系统不是对角占优系统

➤ 引入补偿器 $K = G^{-1}(0)$



```
>> G=[tf(0.1134,[1.78 4.48 1]), tf([0.924],[2.07,1]);  
      tf(0.3378,[0.361,1.09,1]), tf(-0.318,[2.93 1])];  
G1=G; G1.ioDelay=[0.72 0; 0.3 1.29];  
w=logspace(0,1); Kp1=inv(mfrd(G,0)); Kp2=inv([1 0; 0.5 1]);  
H3=mfrd(G1*Kp1*Kp2,w); gershgorin(H3)
```



其他补偿矩阵

➤ 引入补偿矩阵 $K_d(s) = \begin{bmatrix} 1 & 0 \\ 0 & (0.3s + 1)/(0.05s + 1) \end{bmatrix}$

➤ INA绘制 $K_d^{-1}(s) = \begin{bmatrix} 1 & 0 \\ 0 & (0.05s + 1)/(0.3s + 1) \end{bmatrix}$

```
 >> s=tf('s'); Kd=[1 0; 0 (0.3*s+1)/(0.05*s+1)];  
gershgorin(mfrd(G1*Kp1*Kp2*Kd,w))
```

➤ 闭环系统的阶跃响应

```
 >> step(feedback(ss(G1)*Kp1*Kp2*Kd,eye(2)),15)
```




国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第七章 控制器设计的经典方法

多变量频域设计 (下)

Frequency Domain Design of Multivariable Systems (II)



主讲：薛定宇教授



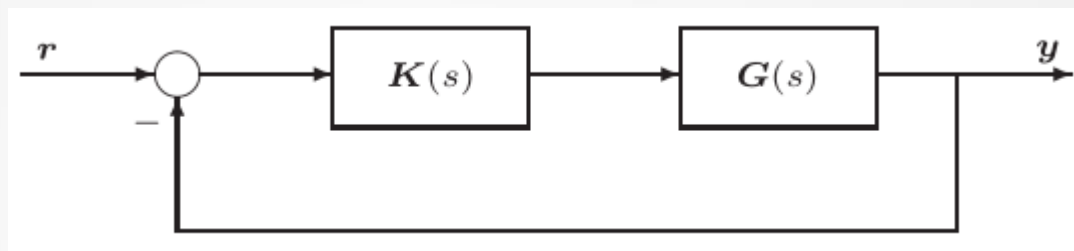
多变量系统的参数最优化设计

- 前面介绍的方法不能完全解耦
- 利用数值最优化技术设计解耦控制器
- 主要参考文献：
 - John Edmunds , Control system design and analysis using closed-loop Nyquist and Bode arrays. International Journal of Control, 1979
- MFD给出直接求解的函数
 - 函数 fedmunds



参数最优化方法的数学基础

➤ 多变量系统框图



- 闭环系统模型 $T(s) = G(s)K(s) \left[I + G(s)K(s) \right]^{-1}$
- 期望在某频率段内闭环模型趋近于 $T_t(s)$
- 目标控制器 K_t 满足 $GK_t = T_t(I - T_t)^{-1}$
- 引入误差 $E = (I - T)(GK - GK_t)(I - T_t)$



数学基础

- 定义最优准则 $\|E\|_2^2 = \min_{N(s)} \int_{-\infty}^{\infty} \text{tr} \left[E^T(-j\omega) E(j\omega) \right] d\omega$
- 选择控制器结构 $K(s) = \frac{N(s)}{d(s)}$
- 选择合适的控制器极点，通过寻优方法搜索出控制器的分子多项式矩阵
- 调用MFD现成函数直接设计

$$N = \text{fedmunds}(w, H, H_t, N_0, D)$$



例7-17 多变量系统设计

➤ 多变量受控对象

$$A = \begin{bmatrix} 0 & 0 & 1.1320 & 0 & -1 \\ 0 & -0.0538 & -0.1712 & 0 & 0.0705 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0.0485 & 0 & -0.8556 & -1.013 \\ 0 & -0.2909 & 0 & 1.0532 & -0.6859 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & 0 & 0 \\ -0.120 & 1 & 0 \\ 0 & 0 & 0 \\ 4.419 & 0 & -1.665 \\ 1.575 & 0 & -0.0732 \end{bmatrix}$$

➤ 模型输入 $C = \begin{bmatrix} I_3 & \mathbf{0}_{3 \times 2} \end{bmatrix}$



```
>> A=[0,0,1.1320,0,-1; 0,-0.0538,-0.1712,0,0.0705; 0,0,0,1,0;  
      0,0.0485,0,-0.8556,-1.013;0,-0.2909,0,1.0532,-0.6859];  
B=[0,0,0; -0.120,1,0; 0,0,0; 4.419,0,-1.665; 1.575,0,-0.0732];  
C=eye(3,5); G=ss(A,B,C,0); w=logspace(-3,2); Hg=mfrd(G,w);
```



目标闭环模型与目标控制器

➤ 选定目标闭环模型

$$T_t(s) = \text{diag} \left[\frac{3^2}{(s+3)^2}, \frac{3^2}{(s+3)^2}, \frac{10^2}{(s+10)^2} \right]$$

➤ 求目标控制器



```
>> s=tf('s'); g=3^2/(s+3)^2;  
T=[g,0,0; 0,g,0; 0,0,10^2/(s+10)^2];
```



```
>> I=eye(3); Kt=inv(G)*T*inv(I-T);  
Hk=mfrd(Kt,w); Ht=mfrd(T,w);  
for i=1:3, for j=1:3,  
    subplot(3,3,3*(i-1)+j); G0=fget(w,Hk,[i,j]);  
    semilogx(w,20*log10(abs(G0))), xlim([0.001,100])  
end, end
```




选择控制器的极点

- 由目标控制器的Bode图选择控制器结构
 - 第一输入无需积分器，其他两个需要
 - 第一输入选择极点 -6
 - 另外两个输入的控制器选择极点 -6、-30
- 控制器结构

$$k_{i1}(s) = \frac{v_{i1}^0 s + v_{i1}^1}{s + 6}, \quad k_{i2}(s) = \frac{v_{i2}^0 s^2 + v_{i2}^1 s + v_{i2}^2}{s(s + 6)}, \quad k_{i3}(s) = \frac{v_{i3}^0 s^2 + v_{i3}^1 s + v_{i3}^2}{s(s + 30)}$$



矩阵选择

➤ 扩展成二阶控制器 $k_{i1}(s) = \frac{0s^2 + v_{i1}^0 s + v_{i1}^1}{0s^2 + s + 6}$

➤ 分子分母矩阵设置

$$D = \begin{bmatrix} 0 & 1 & 6 & 1 & 6 & 0 & 1 & 30 & 0 \\ 0 & 1 & 6 & 1 & 6 & 0 & 1 & 30 & 0 \\ 0 & 1 & 6 & 1 & 6 & 0 & 1 & 30 & 0 \end{bmatrix}$$

$$N = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$



控制器的设计

➤ 输入相关矩阵，直接设计控制器



```
>> d=[0 1 6 1 6 0 1 30 0]; den=[d; d; d];  
    num=[zeros(3,1) ones(3,8)];  
    N=fedmunds(w,Hg,Ht,num,den)
```

➤ 控制器的数学模型



```
>> d1=[1,6]; d2=[1 6 0]; d3=[1 30 0];  
    K11=tf(N(1,2:3),d1); K12=tf(N(1,6),d2);  
    K13=tf(N(1,7:9),d3); K21=tf(N(2,2:3),d1);  
    K22=tf(N(2,5:6),d2); K23=tf(N(2,7:9),d3);  
    K31=tf(N(3,2:3),d1); K32=tf(N(3,6),d2); K33=tf(N(3,7:9),d3);
```



闭环系统的阶跃响应

➤ 控制器的数学模型

$$K(s) = \begin{bmatrix} \frac{-6.5183s - 4.1806}{s + 6} & \frac{1.9101}{s(s + 6)} & \frac{-5.2977s^2 + 6.3218s + 77.927}{s(s + 30)} \\ \frac{-0.7822s + 0.1328}{s + 6} & \frac{9s + 0.7134}{s(s + 6)} & \frac{-0.6161s^2 + 0.6246s + 22.991}{s(s + 30)} \\ \frac{-17.3s - 5.6199}{s + 6} & \frac{5.3316}{s(s + 6)} & \frac{-99.857s^2 - 63.275s + 104.83}{s(s + 30)} \end{bmatrix}$$

➤ 闭环系统的阶跃响应



```
>> K=[K11 K12 K13; K21 K22 K23; K31 K32 K33];  
Gc=feedback(G*K,I); step(Gc,5)
```



多变量系统频域设计小结

- 多变量系统的频域响应与对角占优
 - 现成函数频域响应分析：mfrd()
 - 伪对角占优化：pseuddiag()
- 多变量系统参数最优化设计
 - John Edmunds 算法：fedmunds() 函数
 - 选择目标闭环传递函数，计算目标控制器
 - 由目标控制器特性选择极点和控制结构直接优化



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第七章 控制器设计的经典方法

多变量系统动态解耦

Dynamic Decoupling of Multivariable Systems



主讲：薛定宇教授



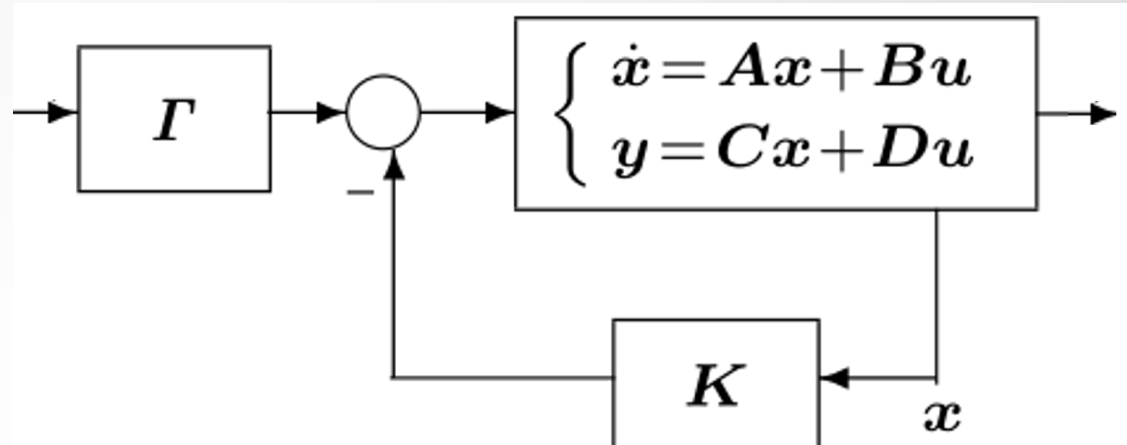
多变量系统的解耦控制

- 解耦的概念与理解，解耦的意义
- 基于状态方程的解耦方法与实现
- 本节主要内容
 - 状态反馈的解耦
 - 标准传递函数
 - 状态反馈的极点配置解耦



状态反馈的解耦

- 开环系统状态方程 (A, B, C, D)
- 引入状态反馈 $u = \Gamma r - Kx$
- 闭环系统的模型



$$G(s) = \left[(C - DK)(sI - A + BK)^{-1} B + D \right] \Gamma$$

- 定义阶次 d_j , 使得 $c_j^T A^{d_j} B \neq 0$
- 其中 , c_j^T 为 C 的第 j 行



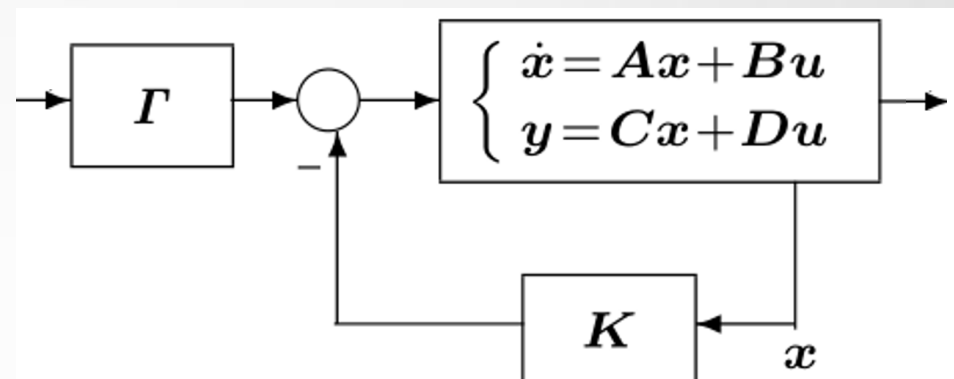
状态反馈解耦

- 如果下面的矩阵非奇异

$$F = \begin{bmatrix} c_1^T A^{d_1} B \\ \vdots \\ c_m^T A^{d_m} B \end{bmatrix}$$

- 则可以设计出状态反馈解耦矩阵

$$\Gamma = F^{-1}, \quad K = \Gamma \begin{bmatrix} c_1^T A^{d_1+1} \\ \vdots \\ c_m^T A^{d_m+1} \end{bmatrix}$$





解耦算法的MATLAB实现

$$F = \begin{bmatrix} c_1^T A^{d_1} B \\ \vdots \\ c_m^T A^{d_m} B \end{bmatrix}$$

➤ 调用格式 $[G_1, K, d, \Gamma] = \text{decouple}(G)$

```
function [G1,K,d,Gam]=decouple(G)
G=ss(G); A=G.a; B=G.b; C=G.c;
[n,m]=size(G.b); F=[]; K0=[];
for j=1:m, for k=0:n-1
    if norm(C(j,:)*A^k*B)>eps, d(j)=k; break; end, end
    F=[F; C(j,:)*A^d(j)*B]; K0=[K0; C(j,:)*A^(d(j)+1)];
end
Gam=inv(F); K=Gam*K0;
G1=minreal(tf(ss(A-B*K,B,C,G.d))*Gam);
```

$$\Gamma = F^{-1}, \quad K = \Gamma \begin{bmatrix} c_1^T A^{d_1+1} \\ \vdots \\ c_m^T A^{d_m+1} \end{bmatrix}$$



例7-19 状态反馈解耦

➤ 受控对象

$$\begin{cases} \dot{x} = \begin{bmatrix} 2.25 & -5 & -1.25 & -0.5 \\ 2.25 & -4.25 & -1.25 & -0.25 \\ 0.25 & -0.5 & -1.25 & -1 \\ 1.25 & -1.75 & -0.25 & -0.75 \end{bmatrix} x + \begin{bmatrix} 4 & 6 \\ 2 & 4 \\ 2 & 2 \\ 0 & 2 \end{bmatrix} u \\ y = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 2 & 0 & 2 \end{bmatrix} x \end{cases}$$

➤ 解耦



```
>> A=[2.25, -5, -1.25, -0.5; 2.25, -4.25, -1.25, -0.25;  
      0.25, -0.5, -1.25, -1; 1.25, -1.75, -0.25, -0.75];  
B=[4, 6; 2, 4; 2, 2; 0, 2]; C=[0, 0, 0, 1; 0, 2, 0, 2];  
G=ss(A,B,C,0); [G1,K,d,Gam]=decouple(G)
```



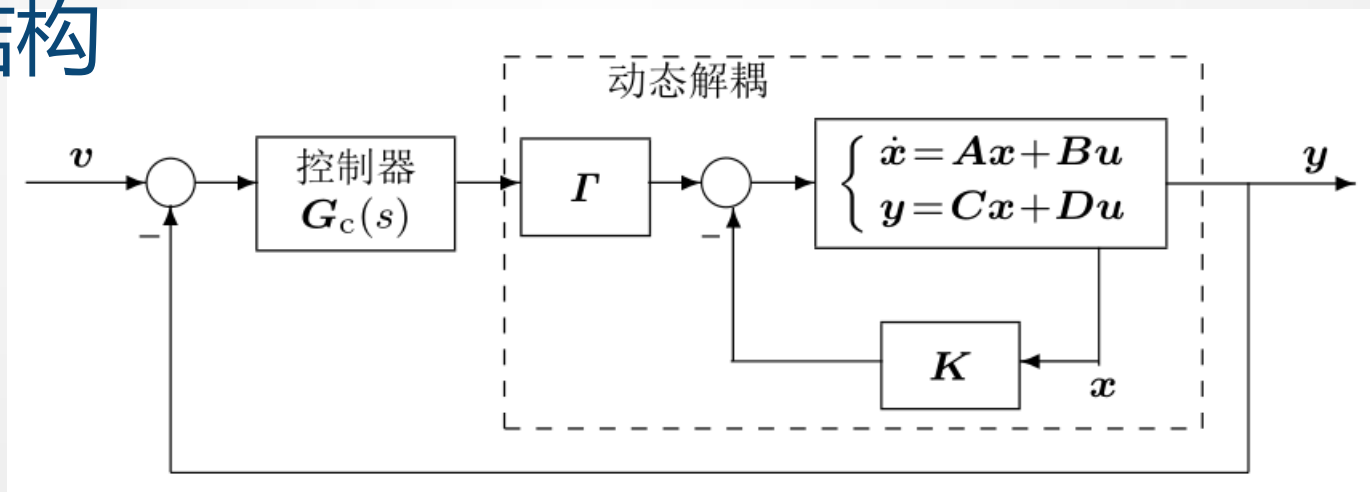
动态解耦的结果

➤ 解耦的结果

$$G_1(s) = \begin{bmatrix} 1/s & 0 \\ -8.9 \times 10^{-16}/s & 1/s \end{bmatrix},$$

$$K = \frac{1}{8} \begin{bmatrix} -1 & -3 & -3 & 5 \\ 5 & -7 & -1 & -3 \end{bmatrix}, \quad \Gamma = \begin{bmatrix} -1.5 & 0.25 \\ 0.5 & 0 \end{bmatrix}$$

➤ 状态反馈结构





标准传递函数

➤ n 阶标准传递函数

$$T(s) = \frac{a_n}{s^n + a_1 s^{n-1} + a_2 s^{n-2} + \cdots + a_{n-1} s + a_n}$$

➤ ITAE最优系数

n	超调量	$\omega_n t_s$	首一化的分母多项式
1			$s + \omega_n$
2	4.6%	6.0	$s^2 + 1.41\omega_n s + \omega_n^2$
3	2%	7.6	$s^3 + 1.75\omega_n s^2 + 2.15\omega_n^2 s + \omega_n^3$
4	1.9%	5.4	$s^4 + 2.1\omega_n s^3 + 3.4\omega_n^2 s^2 + 2.7\omega_n^3 s + \omega_n^4$
5	2.1%	6.6	$s^5 + 2.8\omega_n s^4 + 5.0\omega_n^2 s^3 + 5.5\omega_n^3 s^2 + 3.4\omega_n^4 s + \omega_n^5$
6	5%	7.8	$s^6 + 3.25\omega_n s^5 + 6.6\omega_n^2 s^4 + 8.6\omega_n^3 s^3 + 7.45\omega_n^4 s^2 + 3.95\omega_n^5 s + \omega_n^6$
7	10.9%	10.0	$s^7 + 4.475\omega_n s^6 + 10.42\omega_n^2 s^5 + 15.08\omega_n^3 s^4 + 15.54\omega_n^4 s^3 + 10.64\omega_n^5 s^2 + 4.58\omega_n^6 s + \omega_n^7$



MATLAB实现

➤ 调用格式 $T = \text{std_tf}(\omega_n, n)$

➤ 程序清单

```
function G=std_tf(wn,n)
M=[1,1,0,0,0,0,0 0; 1,1.41,1,0,0,0,0 0;
    1,1.75,2.15,1,0,0,0 0; 1,2.1,3.4,2.7,1,0,0 0;
    1,2.8,5.0,5.5,3.4,1,0 0;
    1,3.25,6.6,8.6,7.45,3.95,1,0;
    1,4.475,10.42,15.08,15.54,10.64,4.58,1];
G=tf(wn^n,M(n,1:n+1).*(wn.^[0:n]));
```



极点配置状态反馈解耦

➤ 定义矩阵 E , 每一行 $e_i^T = c_i^T A^{d_i} B$

➤ 另一个矩阵 F 的第 i 行

$$f_i^T = c_i^T \left(A^{d_i+1} + a_{i,1} A^{d_i} + \cdots + a_{i,d_i+1} I \right)$$

➤ 状态反馈与前置矩阵为 $\Gamma = E^{-1}$, $K = \Gamma F$

➤ 编写极点配置状态解耦的MATLAB函数

$$[G_1, K, d, \Gamma] = \text{decouple_pp}(G, \omega_n)$$



例7-20 前面的系统模型

- 例7-19的系统模型
- 选择期望自然频率 $\omega_n = 5$
- 输入模型并求取解耦控制器



```
>> A=[2.25, -5, -1.25, -0.5; 2.25, -4.25, -1.25, -0.25;  
      0.25, -0.5, -1.25, -1; 1.25, -1.75, -0.25, -0.75];  
B=[4, 6; 2, 4; 2, 2; 0, 2];  
C=[0, 0, 0, 1; 0, 2, 0, 2];  
G=ss(A,B,C,0); [G1,K,d,Gam]=decouple_pp(G,5)
```



得出的结果

➤ 解耦结果

$$\mathbf{G}_1(s) = \begin{bmatrix} 1/(s+5) & 0 \\ \epsilon & 1/(s+5) \end{bmatrix}, \quad \epsilon \approx 10^{-14}$$

➤ 其他矩阵

$$\mathbf{K} = \frac{1}{8} \begin{bmatrix} -1 & 17 & -3 & -35 \\ 5 & -7 & -1 & 17 \end{bmatrix}, \quad \mathbf{\Gamma} = \begin{bmatrix} -1.5 & 0.25 \\ 0.5 & 0 \end{bmatrix}$$

➤ 解耦模型的阶跃响应



```
>> step(G1,10)
```



例7-21 考虑 3×3 多变量系统

➤ 选择 $\omega_n = 3$



```
>> A=[0,0,1.1320,0,-1; 0,-0.0538,-0.1712,0,0.0705;  
      0,0,0,1,0; 0,0.0485,0,-0.8556,-1.013;  
      0,-0.2909,0,1.0532,-0.6859];  
B=[0,0,0; -0.120,1,0; 0,0,0;  
    4.419,0,-1.665; 1.575,0,-0.0732];  
C=eye(3,5); G=ss(A,B,C,0);  
[G1,K,d,Gam]=decouple_pp(G,3)
```



```
>> step(G1,10)
```

➤ 解耦后模型为 $G_1 = \text{diag} \left(\frac{1}{s^2 + 4.23s + 9}, \frac{1}{s + 3}, \frac{1}{s^2 + 4.23s + 9} \right)$



多变量系统解耦小结

- 给出了状态方程系统解耦的系统结构
- 给出了两种多变量状态方程解耦的方法
 - 解耦成对角积分器模型
 - 解耦成对角标准传递函数模型
- 由解耦模型再设计外环控制器可能得出好的控制效果

